Sliding Window ->

↳ In many problem dealing with an array (or a linkedlist), we are asked to find or calculate something among all the contiguous subarray (or sublists) of given size.

1) Subarray / Sublist
   ↳ is a Contiguous part of an array or lists.
   ↳ Order is preserved.

2) Subsequence
   ↳ A subsequence is a sequence derived by deleting some (or no) element, without changing the order.
   ↳ Element do not need to be contiguous.

3) Subset
   ↳ A Subset refer to any combination of elements.
   ↳ Often used in set theory.

1) Avg of all Contiguous subArray of size `K` in it.

```
#include <bits/stdc++.h>

class AverageOfSubarrayOfSizeK {
    vector<double> findAverages(int K, vector<int> &arr){
        vector<double> result();

        double windowSum = 0;

        int windowStart = 0;

        for(int windowEnd = 0; windowEnd < arr.size(); windowEnd++){
            windowSum += arr[windowEnd];

            if(windowEnd >= K-1){   // this will cause segmentation fault
                result[windowStart] = windowSum/K;
```

result.push_back (windowSum/K); ← use

```
                windowSum -= arr[windowStart];
                windowStart++;
            }
        }
        return result;
    }
};
```

Problem - Max SubArray of Size K

Problem Statement

Given an array of positive numbers and a positive number ‘k’, find the **maximum sum of any contiguous subarray of size ‘k’.**

Example 1:

```
Input: [2, 1, 5, 1, 3, 2], k=3
Output: 9
Explanation: Subarray with maximum sum is [5, 1, 3].
```

Example 2:

```
Input: [2, 3, 4, 1, 5], k=2
Output: 7
Explanation: Subarray with maximum sum is [3, 4].
```

Try it yourself

Try solving this question here:

```
#include <bits/stdc++.h>
using namespace std;
    int findMaxSumSubArray(int k, vector<int> &arr){
        int windowSum = 0;
        int maxSum = 0;
        int windowStart = 0;
        for(int windowEnd = 0; windowEnd < arr.size(); windowEnd++){
            windowSum += arr[windowEnd];
            if(windowEnd >= K-1){
                maxSum = max(maxSum, windowSum);
                windowSum -= arr[windowStart];
                windowStart++;
            }
        }
```

```
        return maxSum;
    }
};
```

Problem - Smallest Subarray with a given Sum →

Problem Statement

Given an array of positive numbers and a positive number ‘S’, find the length of the **smallest contiguous subarray whose sum is greater than or equal to ‘S’.** Return 0, if no such subarray exists.

Example 1:

```
Input: [2, 1, 5, 2, 3, 2], S=7
Output: 2
Explanation: The smallest subarray with a sum great than or equal to '7' is [5, 2].
```

```
#include <bits/stdc++.h>
using namespace std;
    int findMinSubArray(int S, vector<int>& arr){
        int windowSum = 0;
        minLength = numeric_limits<int>::max();
        int windowStart = 0;
        for(int windowEnd = 0; windowEnd < arr.size(),windowEnd++){
            windowSum += arr[windowEnd];
            while(windowSum >= S){
                minLength = min(minLength, windowEnd-windowStart+1);
                windowSum -= arr[windowStart];
                windowStart++;
            }
        }
    return minLength == numeric_limits<int>::max()?0:minLength;
}
```

when to use if inside while

↳ when you only need to check once whether the window is valid ie that the condition (eg size, sum, character count) rarely breaks & does'nt need multiple consecutive shrinking

when to use while inside while

↳ When you repeatedly shrink the window until it become valid again.

↳ This happens in problems where you must enforce a strict constraint (like sum ≤ K, unique char ≤ K).

Problem → Longest Substring with K Distinct Characters →

Given a string, find the length of the **longest substring** in it **with no more than K distinct characters.**

Example 1:

```
Input: String="araaci", K=2
Output: 4
Explanation: The longest substring with no more than '2' distinct characters is "araa".
```

Example 2:

```
Input: String="araaci", K=1
Output: 2
Explanation: The longest substring with no more than '1' distinct characters is "aa".
```

Example 3:

? Ask YourSelf

↳ Do I need to track count or occurences of element in the window ?

↳ if yes then you likely need a hash map.

Common Scenarios →

count of distinct element in window	→ You need to track frequency of each element.
character frequency match (like anagram)	→ You must match the exact freq
Tracking the first or last occurence of a character	→ Need to map element to positions.

Maintaing a sliding window of constraints like "at most K distinct characters"
→ You need to know how many unique element are in the current Window

Problems involving substring with constraints (eg, "Longest substring without repeating characters")
→ You need to track if a character has already appeared in the window

```
#include <bits/stdc++.h>
using namespace std;
    int findLength( string &str, int K){
    int windowStart = 0;
    int maxLength = 0;
    unordered_map<char, int> charFrequencyMap;
for( int windowEnd = 0; windowEnd < str.size(); windowEnd++){
    char rightChar = str[windowEnd];
    charFrequencyMap[rightChar]++;
    while (charFrequencyMap.size() > K) {
    char leftChar = str[windowStart];
    charFrequencyMap[leftChar]--;
    if( charFrequencyMap[rightChar] == 0){
        charFrequencyMap.erase( leftChar);
    }
    windowStart++;
}
maxLength = max(maxLength, windowEnd-windowStart+1);
}
```

return maxLength;
}

Problem - Fruits into Baskets.

Problem Statement

Given an array of characters where each character represents a fruit tree, you are given **two baskets** and your goal is to put **maximum number of fruits in each basket.** The only restriction is that **each basket can have only one type of fruit.**

You can start with any tree, but once you have started you can't skip a tree. You will pick one fruit from each tree until you cannot, i.e., you will stop when you have to pick from a third fruit type.

Write a function to return the maximum number of fruits in both the baskets.

Example 1:

```
Input: Fruit=['A', 'B', 'C', 'A', 'C']
Output: 3
Explanation: We can put 2 'C' in one basket and one 'A' in the other from the subarra
y ['C', 'A', 'C']
```

Example 2:

```
Input: Fruit=['A', 'B', 'C', 'B', 'B', 'C']
Output: 5
Explanation: We can put 3 'B' in one basket and two 'C' in the other basket.
This can be done if we start with the second letter: ['B', 'C', 'B', 'B', 'C']
```

```cpp
#include <bits/stdc++.h>
using namespace std;
    int findLength(vector<int>& arr){
        int windowStart = 0;
        int maxLength = 0;
        unordered_map<char, int> fruitFrequencyMap;
        for(int windowEnd = 0; windowEnd < arr.size(); windowEnd++){
            fruitsFrequencyMap[arr[windowEnd]]++;
            while(fruitsFrequencyMap.size() > 2){
                fruitsFrequencyMap[arr[windowStart]]--;
                if(fruitsFrequencyMap[arr[windowStart]]==0){
                    fruitsFrequencyMap.erase(arr[windowStart]);
                }
                windowStart++;
```

```
        }
        maxLength = max(maxLength, windowEnd - windowStart+1);
    }
    return maxLength;
}
```

↳ Similar Problems (Longest Substrings with at most 2 distinct Characters).

Problem - No - repeat Substring

Given a string, find the **length of the longest substring** which has **no repeating characters.**

Example 1:

```
Input: String="aabccbb"
Output: 3
Explanation: The longest substring without any repeating characters is "abc".
```

Example 2:

```
Input: String="abbbb"
Output: 2
Explanation: The longest substring without any repeating characters is "ab".
```

Example 3:

```
Input: String="abccde"
Output: 3
Explanation: Longest substrings without any repeating characters are "abc" & "cde".
```

Try it yourself #

Try solving this question here:

```
#include <bits/stdc++.h>
using namespace std;
    int findLength(string& str){
        int windowStart = 0;
        int maxLength = 0;
        unordered_map<char, int> charIndexMap;
        for(int windowEnd = 0; windowEnd<str.size(); windowEnd++){
            char rightChar = str[windowEnd];
            if(charIndexMap.find(rightChar) != charIndexMap.end()){
                windowStart = max(windowStart, charIndexMap[rightChar]+1);
            }
            charIndexMap[rightChar] = windowEnd;
            maxLength = max(maxLength, windowEnd - windowStart+1);
```

```
    }
    return maxLength;
}
```

Problem - Longest Substring with Same Letters after Replacement

Problem Statement

Given a string with lowercase letters only, if you are allowed to **replace no more than 'k' letters** with any letter, find the **length of the longest substring having the same letters** after replacement.

Example 1:

```
Input: String="aabccbb", k=2
Output: 5
Explanation: Replace the two 'c' with 'b' to have a longest repeating substring "bbbbb".
```

```
#include<bits/stdc++.h>
using namespace std;
int findLength(string &str, int K){
    int windowStart = 0;
    int maxLength = 0;
    int maxRepeatLetterCount = 0;
    unordered_map<char,int> letterFrequencyMap;
    for(int windowEnd =0; windowEnd < str.length(); windowEnd++){
        char rightChar = str[windowEnd];
        letterFrequencyMap[rightChar]++;
        maxRepeatLetterCount = max(maxRepeatLetterCount,letterFrequencyMap[rightchar]);
        if(windowEnd - windowStart +1- maxRepeatLetterCount > K){
            char LeftChar = str[windowStart];
            LetterFrequencyMap[leftChar]--;
            windowStart++;
        }
        maxLength = max(maxLength, windowEnd - windowStart +1);
    }
    return maxLength;
}
```

Problem- Longest Subarray with Ones after Replacement.

Problem Statement

Given an array containing 0s and 1s, if you are allowed to **replace no more than 'k' 0s with 1s**, find the length of the **longest contiguous subarray having all 1s.**

Example 1:

```
Output: 6
Explanation: Replace the '0' at index 5 and 8 to have the longest contiguous subarray of 1
s having length 6.
```

Example 2:

```
Input: Array=[0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1], k=3
Output: 9
Explanation: Replace the '0' at index 6, 9, and 10 to have the longest contiguous subarra
y of 1s having length 9.
```

```
#include <iostream>
using namespace std;

int findLength(vector<int>&arr, int K){
int windowStart = 0;
int maxLength= 0;
int maxOnesCount =0;
// try to Extend the range [windowStart, windowEnd]
for( int windowEnd= 0; windowEnd < arr.size(); windowEnd++){
    if(arr[windowEnd] == 1){
        maxOnesCount++;
    }
    // current window size is from windowStart to windowEnd, overall we
    have a maximum of 1's repeating a maximum of 'maxOnesCount' times
    this means that we can have a window with 'maxOnesCount' 1s and the
    remaining are 0s which should replace with 1s.
    // now, if the remaining 0s are more than 'K', it is the time to shrink
     the window as we are not allowed to replaced more than 'K' 0s.
    if( windowEnd - windowStart + 1 - maxOnesCount > K){
        if( arr[windowStart] == 1){
            maxOnesCount--;
        }
        windowStart++;
    }
    maxLength = max(maxLength, windowEnd - windowStart + 1);
}
```

return maxLength;
}

Problem - Permutation of a String

example, “abc” has the following six permutations:

1. abc
2. acb
3. bac
4. bca
5. cab
6. cba

If a string has ‘n’ distinct characters it will have $n!$ permutations.

Example 1:

```
Input: String="oidbcaf", Pattern="abc"
Output: true
Explanation: The string contains "bca" which is a permutation of the given pattern.
```

```
#include <bits/stdc++.h>
using namespace std;

class StringPermutation {

  public:
    static bool findPermutation(string &str, string &pattern){

        int windowStart = 0;
        int matched = 0;

        unordered_map<char, int>charFrequencyMap;
        for(auto chr : pattern) {
            CharFrequencyMap[chr]++;
        }

    for(int windowEnd = 0 ; windowEnd < str.length(); windowEnd++){
        char rightChar = str[windowEnd];
        if (charFrequencyMap.find(rightChar) != charFrequencyMap.end()){

            charFrequencyMap[rightChar]--;
            if( charFrequencyMap[rightChar] == 0) {
                matched--;
            }
            charFrequencyMap[leftChar]++;
        }
    }
}
```

```
        return false;
    }
};
```

Problem - String Anagram.

String Anagrams (hard)

Given a string and a pattern, find **all anagrams of the pattern in the given string.**

Anagram is actually a **Permutation** of a string. For example, "abc" has the following six anagrams:

1. abc
3. bac
4. bca
5. cab
6. cba

Write a function to return a list of starting indices of the anagrams of the pattern in the given string.

Example 1:

```
Input: String="ppqp", Pattern="pq"
Output: [1, 2]
Explanation: The two anagrams of the pattern in the given string are "pq" and "qp".
```

Example 2:

```
Input: String="abbcabc", Pattern="abc"
Output: [2, 3, 4]
Explanation: The three anagrams of the pattern in the given string are "bca", "cab", and "abc".
```

```
#including <luits/stdc++.h>
using namespace std;

class StringAnagrams{
  public:
    static vector<int> findStringAnagram(string &str, string &pattern){
        int windowStart=0;
        int matched=0;
        unordered_map<char, int> charFrequencyMap;
        for(auto chr: pattern){
            charFrequencyMap[chr]++;
        }
        vector<int> resultIndices;
        for(int windowEnd=0; windowEnd<str.length(); windowEnd++){
            char rightChar=str[window];
            if(charFrequencyMap.find(rightChar)!=charFrequencyMap.end()){
                charFrequencyMap[rightChar]--;
                if(charFrequencyMap[rightChar]==0){
```

```
                matched++;
            }
        }

        if (match == (int) charFrequencyMap.size()){
            resultIndices.push_back(windowStart);
        }
        if(windowEnd >= pattern.length() - 1) {
            char leftChar = str[windowStart++];
            if( charFrequencyMap.find(leftChar) != charFrequencyMap.end()){
                if ( charFrequencyMap[leftChar] == 0){
                    matched--;
                }
                charFrequencyMap[leftChar]++;
            }
        }
    }
    return resultIndices;
    }
};
```

Problem - Smallest Window containing Substring (hard)

Smallest Window containing Substring (hard)

Given a string and a pattern, find the **smallest substring** in the given string which has **all the characters of the given pattern.**

Example 1:

```
Input: String="aabdec", Pattern="abc"
Output: "abdec"
Explanation: The smallest substring having all characters of the pattern is "abdec"
```

Example 2:

```
Input: String="abdabca", Pattern="abc"
Output: "abc"
Explanation: The smallest substring having all characters of the pattern is "abc".
```

Example 3:

```
Input: String="adcad", Pattern="abc"
Output: ""
Explanation: No substring in the given string has all characters of the pattern.
```